AI Compiler Performance Engineering

Auto-Loops Books — ch21

Contents

1 Unified AI Compiler Analysis and Cross-Hardware Benchmark 1

1.1 Compiler ratings 1
1.2 Compile overhead 4
1.3 Hardware binding 7
1.4 Search space 10
1.5 YiRage benchmark 12
1.6 Key Takeaways 17
1.7 Conclusion 18

Bibliography 19

Index 22

Chapter 1

Unified AI Compiler Analysis and Cross-Hardware Benchmark

Each compiler in this Part advertises its own headline number on its own favored workload, which makes them almost impossible to compare honestly. This chapter does the comparison the projects do not: it rates TVM, Triton, IREE, Glow, and Mirage on one axis that actually predicts interactive serving cost—how well each collapses a token's work into few, resident kernel launches—rather than on peak GEMM throughput (Chen *et al.*, 2018b; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024). The yardstick is the one Chapter 1 set: dense Llama-3.2-1B averages 847.5 GPU launches per output token on H100 and OLMoE-1B/7B averages 9,305, so the metric that matters is launches/token and bytes/token at batch-1, not datasheet FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026). The point is not to crown a winner but to show that these stacks occupy different niches, and that the right choice is a routing decision—which is exactly what the book's YiRage compiler encodes (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

1.1 Compiler ratings

A useful **compiler rating** for decode is not a single score; it is a profile across five axes: decode-fusion strength, launch collapse, multi-hardware breadth, search/compile cost, and correctness guarantee (Davies *et al.*, 2025). Rated this way the stacks separate cleanly. TVM is broad and search-driven but graph-and-TensorIR-staged; Triton is a GPU block compiler with hand-placed fusion boundaries; IREE is the

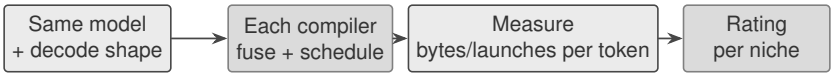


Figure 1.1: A fair comparison rates each **compiler** on the same decode metrics—bytes/token and launches/token—instead of each project's favored microbenchmark.

Table 1.1: Decode-oriented **rating** of the surveyed compilers; "launch collapse" is the batch-1 lever and the column that most predicts ms/token (Vellaisamy *et al.*, 2026; Chen, 2026).

Compiler	Primary niche	Launch-collapse mechanism
TVM	Broad, search-driven	Fusion + graph executor / AOT
Triton	GPU block kernels	Hand-placed fusion + CUDA Graphs
IREE	Widest HW matrix	Whole-program stream scheduling
Glow	Edge / CPU	AOT bundle (no launch loop)
Mirage	GPU superoptimizer	Searched megakernel

widest hardware matrix with whole-program scheduling; Glow is the edge/CPU specialist; Mirage is the superoptimizer that discovers fusion and verifies it (Chen *et al.*, 2018b; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024).

It is worth being explicit about why the niches are real and not marketing. The differences trace to architectural commitments made early in each project. TVM committed to a learned-cost-model search over a graph-plus-TensorIR split, which buys breadth and automation at the cost of a staged lowering that can leave fusion on the table (Chen *et al.*, 2018b; Zheng *et al.*, 2020). Triton committed to a Python tile abstraction over a GPU SIMT machine, which buys productivity and precise GPU control but leaves the fusion boundary to the author and excludes static NPUs (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). IREE committed to whole-program compilation through an MLIR dialect stack and a hardware abstraction layer, which buys the widest matrix and compiled host scheduling at the cost of hiding per-target residency (IREE / iree-org contributors, 2024). Glow committed to a two-level IR and AOT bundles, which buys a predictable edge footprint by giving up datacenter-GPU depth (Rotem *et al.*, 2018). Mirage committed to multi-level superoptimization with verification, which buys reachable megakernels at the cost of search time (Wu *et al.*, 2024).

The rating exposes a recurring truth: a stack can score high on isolated kernel quality and low on launch collapse, and at batch-1 the second column is the one that moves p99 (Vellaisamy *et al.*, 2026; NVIDIA, 2024). On a GPU the

launch-collapse mechanisms differ—Triton relies on the author to widen the fused region then captures it, IREE schedules the whole program, Mirage searches for the megakernel—but all three are attacking the same orchestration tax (Chen, 2026; Mirage Project contributors, 2025). The mistake a team makes is to read one project's headline speedup as a global ranking; the speedups are measured against different baselines on different workloads, and only a common decode harness makes them comparable (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

1.1.1 Rating methodology

The rating methodology must fix the variables that published benchmarks leave free: the model, the checkpoint, the tokenizer, the context buckets, the batch size, the driver and runtime versions, and the metric definitions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Every compiler under test should receive the same reference program and be asked to produce its best artifact for the same decode shape (Chen, 2020; Davies *et al.*, 2025). The harness then measures each artifact on the same three counters and publishes the configuration with the numbers, because a benchmark without a pinned configuration is a claim without an experiment (Chen, 2026; Vellaisamy *et al.*, 2026). The methodology should also state what each compiler was allowed to tune: a search-based stack gets its search budget, an AOT stack gets its compile time, and the budget should be proportional to the deployment model the comparison is meant to inform (Wu *et al.*, 2024; Zheng *et al.*, 2020). Without this discipline, the rating measures the benchmark authors' generosity rather than the compilers.

1.1.2 What the columns mean operationally

The five rating axes translate directly into deployment questions. Decode-fusion strength asks whether the stack can keep a layer's intermediates on-chip at batch-1; launch collapse asks how many host dispatches survive per token; multi-hardware breadth asks whether one program covers the fleet; search and compile cost asks how long a new model takes to deploy; and correctness guarantee asks what confidence the artifact carries (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). A team choosing a compiler for one product should weight these columns by their actual constraints rather than by an average score (Chen, 2020; Davies *et al.*, 2025). A single-SKU GPU serving team weights launch collapse and correctness highest; a fleet with many device types weights breadth; a product that recompiles nightly weights compile cost (Kwon *et al.*, 2023; Davies *et al.*, 2025). The rating table is therefore a decision aid with weights supplied by the reader, not a leaderboard

with a universal winner (Vellaisamy *et al.*, 2026; Chen, 2026).

1.1.3 Reading a rating table

Rating tables mislead when read as ranks instead of profiles (Davies *et al.*, 2025; Chen, 2026). A cell that says a compiler has weak multi-hardware breadth is not a bug in that compiler; it may be the deliberate price of deep GPU residency reasoning (OpenAI / triton-lang contributors, 2024; Wu *et al.*, 2024). The reader should locate the workload's constraints in the table first and then identify the stacks whose strengths match those constraints (Chen, 2020; Vellaisamy *et al.*, 2026). The table should also carry the evidence for each cell—which artifact, which bucket, which benchmark produced the score—so a disputed cell can be re-run rather than argued (Davies *et al.*, 2025; Chen, 2026). When two stacks score equally on the columns that matter, choose by operational factors outside the table: team skill, ecosystem integration, and maintenance burden (Chen, 2020; Davies *et al.*, 2025). A rating is a starting point for a routing decision, never the decision itself (Vellaisamy *et al.*, 2026; Chen, 2026).

1.1.4 Baseline discipline

Every compiler rating needs an explicit baseline measured in the same harness, because a speedup without a baseline definition is uninterpretable (Davies *et al.*, 2025; Chen, 2026). The baseline should be the stack a team would actually use without the compiler under test—usually eager PyTorch or a vendor library path—measured with the same warm-up, buckets, and counters (Vellaisamy *et al.*, 2026; Chen, 2020). Published speedups from each project should be re-derived against this common baseline rather than copied, because their original baselines differ (Wu *et al.*, 2024; Zheng *et al.*, 2020; Mirage Project contributors, 2025). The rating table should show absolute counters alongside speedup ratios so a 2x win on a bad baseline is visible for what it is (Chen, 2026; Davies *et al.*, 2025). Baseline discipline is the unglamorous work that makes every other column of the comparison trustworthy (Chen, 2020; Vellaisamy *et al.*, 2026).

1.2 Compile overhead

The second axis is **compile overhead**, and it is the cost teams under-budget. The stacks pay in different places. TVM's Ansor and AutoTVM pay a measured search—thousands of on-device trials per operator—reporting up to 3.8× (Ansor,



Figure 1.2: Where each stack pays its **compile** cost: JIT autotuning, ahead-of-time compilation, or superoptimization search.

Intel CPU) and 1.2–3.8× (AutoTVM) in exchange (Zheng *et al.*, 2020; Chen *et al.*, 2018a). Triton pays a per-shape autotune warmup the first time a kernel key is seen, so the first request after a shape change is slow (OpenAI / triton-lang contributors, 2024). IREE and Glow pay their cost ahead of time, producing a deployable artifact with no runtime compile (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). Mirage pays the largest one-time cost—superoptimization search plus probabilistic verification—and reports 1.1–2.9×, with its MPK megakernel reaching 1.2–6.7× (Wu *et al.*, 2024; Mirage Project contributors, 2025).

The shape of the overhead also differs in ways that matter operationally, not just in total cost. A JIT autotuner's cost is incremental and amortizing: it is paid lazily, per new shape, and warm shapes are free—but it makes the first request after any shape change slow, which is visible to a user at exactly the wrong moment (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). An AOT compile front-loads everything into the build, so runtime latency is predictable and there is no first-request penalty, at the cost of a longer build and a larger artifact that must be regenerated when the model or target changes (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). A superoptimization search is the most expensive but also the most one-time: once a megakernel is found and verified for a model-target pair, it serves every subsequent token with no further search (Wu *et al.*, 2024; Mirage Project contributors, 2025). A team that values predictable tail latency leans AOT or pre-searched; a team iterating rapidly on shapes tolerates JIT.

The engineering rule that falls out is to match the **overhead** model to the deployment. A served LLM whose decode layers are reused across billions of tokens justifies an expensive one-time search or AOT compile, because the cost amortizes; a rapidly changing or one-off graph does not, and there a JIT autotuner's incremental cost or a lighter pass pipeline wins (Kwon *et al.*, 2023; Davies *et al.*, 2025). The **compile**-time/runtime-latency trade is itself a design decision, not an afterthought.

1.2.1 Budgeting compile cost in the release pipeline

Compile overhead must be budgeted in the release pipeline, not discovered at deployment time (Chen, 2020, 2026). The budget has three components that fail

differently: wall-clock compile time per artifact, on-device trial time for autotuners, and verification or numerical-test time for searched kernels (Zheng *et al.*, 2020; Wu *et al.*, 2024; Davies *et al.*, 2025). Each component belongs in the release manifest so a model change that inflates compile time is visible before it delays a rollout (Chen, 2020; Vellaisamy *et al.*, 2026). The pipeline should also separate build-time compilation from serving-time fallback: any compilation that can happen in CI should happen there, leaving the runtime with a warm cache and no first-request penalty (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). Teams that skip this budgeting treat compile time as an externality and discover it during the first production rollout (Chen, 2026; Davies *et al.*, 2025).

1.2.2 First-token latency and the compile contract

The user-visible cost of compilation is the first-token latency it produces (Kwon *et al.*, 2023; Davies *et al.*, 2025). A JIT stack that autotunes on the first request converts a cold start into a user-visible stall; an AOT or pre-searched stack pays the same cost at build time and serves every token predictably (OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024). The product contract should state where compile cost is allowed to appear: never in steady-state serving, and only in bounded, predictable places during warmup or model load (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). When a stack cannot meet that contract, the deployment must precompile or pre-warm the exact shapes the product will serve (Kwon *et al.*, 2023; Chen, 2020). Measuring first-token latency under a cold cache, warm cache, and pre-compiled artifact is therefore part of the compiler benchmark, not an optional operational nicety (Chen, 2026; Davies *et al.*, 2025).

1.2.3 When expensive compilation pays

The amortization question determines whether a search-based or AOT compiler is worth its cost (Wu *et al.*, 2024; Kwon *et al.*, 2023). A decoder layer that runs once per token across many concurrent users accrues billions of executions, so even a multi-hour search costs a negligible fraction of a nanosecond per execution (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). A preprocessing graph that runs once per model version does not justify the same expense, and a lighter compiler or a direct kernel library is the rational choice (Chen, 2020, 2026). The break-even calculation should include artifact storage, verification, and regeneration frequency, because a model that changes weekly forces the search cost weekly (Wu *et al.*, 2024; Davies *et al.*, 2025). Publishing the break-even arithmetic with the benchmark makes the compile-cost axis a decision rather than a preference (Vellaisamy *et al.*,

2026; Chen, 2026).

1.2.4 Measuring compile overhead in CI

Compile overhead should be measured in the same CI that builds the artifacts, with a separate report that tracks wall-clock time, trial time, artifact size, and verification time per configuration (Chen, 2020, 2026). The report makes regressions visible: a compiler upgrade that doubles search time or an input change that inflates artifact size shows up before it delays a release (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The measurement should run on representative hardware because on-device autotuning time does not correlate with build-host CPU time (Zheng *et al.*, 2020; OpenAI / triton-lang contributors, 2024). Budgets attached to the report can fail the build when compile cost exceeds the product's deployment tolerance (Chen, 2026; Davies *et al.*, 2025). This turns the compile-overhead axis from an anecdote into a gated metric that the release process enforces (Chen, 2020; Vellaisamy *et al.*, 2026).

1.2.5 Warm caches and build reuse

Compile overhead accounting must distinguish cold builds from warm builds, because caches change the real cost dramatically (Chen, 2020; OpenAI / triton-lang contributors, 2024). A CI job with a warm autotune cache and reusable search results measures the steady-state cost of a model change, while a cold cache measures the worst-case onboarding cost (Zheng *et al.*, 2020; Wu *et al.*, 2024). Both numbers belong in the report, since a team iterating on a model sees the warm number and a new team onboarding sees the cold one (Davies *et al.*, 2025; Chen, 2026). Build reuse policies—which artifacts can be cached across model versions and which must regenerate—should be explicit and versioned (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The distinction keeps compile-cost budgets honest across the life of a project (Chen, 2026, 2020).

1.3 Hardware binding

The third axis is how each stack performs **hardware binding**—how late and how flexibly it commits to a target. TVM binds through target strings and forks schedules per backend; Triton binds to a GPU through its TritonGPU dialect and PTX/AMDGCN codegen and does not target static NPUs; IREE binds latest, through a Vulkan-inspired HAL that abstracts CPU, GPU, and more behind one

interface; Glow binds through LLVM and cross-compiles to edge cores; Mirage binds to the GPU compute hierarchy by construction (Chen *et al.*, 2018b; OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024).

Binding time is really a question of where the abstraction boundary sits between the program and the silicon. TVM's target strings bind at schedule-selection time, so the same compute is re-scheduled per backend with a fork in the pass pipeline (Chen *et al.*, 2018b). IREE's HAL pushes the boundary as late as possible—the dispatch regions are formed once and only the executable translation is per-target—so one program description fans out to many devices (IREE / iree-org contributors, 2024). Glow binds through LLVM, inheriting LLVM's target support for free and gaining cross-compilation to embedded cores as a side effect (Rotem *et al.*, 2018). Triton and Mirage bind earliest and narrowest, to a GPU machine model, which is why their reasoning about shared memory and launches is the most precise and least portable (Tillet *et al.*, 2019; Wu *et al.*, 2024).

The decode consequence of late versus early **binding** is concrete. A late-binding HAL like IREE's makes a model portable across a wide matrix but hides per-target residency decisions that only batch-1 telemetry exposes, so a green build is not a green decode (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). An early, narrow binding like Triton's or Mirage's gives up breadth but lets the compiler reason precisely about one machine's shared memory and launch model, which is why the deepest decode wins are GPU-specific (Chen, 2026; SemiAnalysis, 2024). Across all of them the hardware truth is unchanged: GPUs are bound by shared-memory residency, CPUs by cache, NPUs by static tile/DMA plans, so no single binding is optimal everywhere (NVIDIA, 2022; AMD, 2024b,a).

1.3.1 Binding scenarios

Hardware binding becomes concrete through three scenarios. A datacenter GPU workload binds early and deeply when the team wants shared-memory-aware scheduling, choosing Triton or Mirage and accepting that the artifact is GPU-specific (OpenAI / triton-lang contributors, 2024; Wu *et al.*, 2024). A heterogeneous fleet binds late through an abstraction like IREE's HAL so one program covers GPU, CPU, and accelerator targets, accepting that per-target residency must be validated separately (IREE / iree-org contributors, 2024; AMD, 2024a). An edge product binds through an AOT path like Glow's LLVM backend so the same source cross-compiles to the exact embedded core, accepting a fixed schedule per target (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Each scenario is a different answer to the same question: how much portability does the product need, and how much

depth is it willing to trade for it (Davies *et al.*, 2025; Chen, 2026). The scenario frame prevents binding discussions from becoming abstract ideology (Vellaisamy *et al.*, 2026; Chen, 2020).

1.3.2 Validation burden of binding

The later a stack binds, the more per-target validation the deployment owes (IREE / iree-org contributors, 2024; AMD, 2024a). A single portable artifact that lowers to many devices still needs residency and byte validation on each device, because the compiler cannot prove decode legality across targets from one build (Chen, 2026; Davies *et al.*, 2025). An early-bound compiler narrows the validation surface but concentrates it: one GPU artifact must be validated for every SKU generation the fleet runs (NVIDIA, 2022; Wu *et al.*, 2024). The validation matrix should therefore be derived from the binding choice: late binding produces a wide but shallow matrix that grows with targets, while early binding produces a deep but narrow matrix that grows with SKUs (Chen, 2020; Vellaisamy *et al.*, 2026). Compile reports, byte ledgers, and numerical checks are the artifacts that populate either matrix (Chen, 2026; Davies *et al.*, 2025). Teams that bind late without building the per-target validation matrix have only delayed the work, not avoided it (IREE / iree-org contributors, 2024; Chen, 2026).

1.3.3 Per-target residency proof

Whatever the binding time, every deployment cell needs a per-target residency proof: the artifact must show that its fused regions fit the target's on-chip budgets and that the measured bytes/token matches the schedule's intent (Chen, 2026; Davies *et al.*, 2025; AMD, 2024a). The proof has three parts: a compile-time report of buffers and capacities, a runtime measurement of bytes and launches, and a numerical check against the reference (Chen, 2020, 2026). Early-bound compilers can generate most of the proof from their machine model; late-bound compilers must generate it per target at validation time (Wu *et al.*, 2024; IREE / iree-org contributors, 2024). A cell without all three parts is not a supported deployment, it is an experiment (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The binding debate therefore resolves into a simple accounting question: who owns the per-target proof and when is it generated (Chen, 2026, 2020).

1.3.4 Device-class cells

The hardware binding comparison is cleanest when expressed in device-class cells rather than vague categories (Davies *et al.*, 2025; Chen, 2026). A datacenter GPU cell specifies the GPU generation, the context buckets, and the batch-1 shape; an edge CPU cell specifies the core, the SRAM budget, and the power envelope (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024a). Each stack earns a rating per cell, and the rating may change between cells of the same class because shared-memory capacity and cache size differ (Chen, 2020, 2026). The cell definition forces the comparison to state the machine model each compiler reasoned against, which is the real subject of the binding axis (Wu *et al.*, 2024; IREE / iree-org contributors, 2024). Device-class cells also make the validation matrix concrete: each cell lists the compilers that can serve it and the artifacts that prove it (Vellaisamy *et al.*, 2026; Chen, 2026).

1.4 Search space

The fourth axis is the **search** space each stack explores and how it is navigated, because that determines what optimizations are even reachable. AutoTVM searches hand-written schedule templates with a learned cost model; Ansor removed the templates and samples a hierarchical space, **autotun**ing with evolutionary search and a task scheduler that spends budget where it moves end-to-end latency (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Triton's autotuner explores only the configuration list the author supplies—leaner, but bounded by human imagination (OpenAI / triton-lang contributors, 2024). Mirage searches the largest space of all: a multi-level μGraph that can invent new custom kernels, pruned by abstraction and certified by probabilistic equivalence (Wu *et al.*, 2024).

There is a spectrum here from human-specified to machine-discovered optimization, and it correlates with the verification burden. At the human-specified end, Triton's config list and AutoTVM's templates are trustworthy by construction—the schedules are variations a human vetted—but they cannot exceed human imagination (OpenAI / triton-lang contributors, 2024; Chen *et al.*, 2018a). At the machine-discovered end, Mirage's superoptimizer can synthesize a kernel no one wrote, which is powerful precisely because it is not bounded by templates, but that power forces the question of correctness: a discovered kernel must be proven equivalent, which is why probabilistic equivalence verification is not optional for it (Wu *et al.*, 2024). Ansor sits in between, sampling a structured space without explicit templates but within a fixed schedule grammar (Zheng *et al.*, 2020).

The reachability difference is the point. A template-bounded **search** can only find schedules a human anticipated, so it cannot discover a megakernel; a superoptimizer's search can, which is why Mirage and MPK reach fusions the others structurally cannot (Wu *et al.*, 2024; Mirage Project contributors, 2025). The cost, as the overhead axis noted, is search time—so the right **autotun**ing strategy depends on how reused the program is and how much of the optimization you are willing to let the compiler discover rather than specify (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

1.4.1 Choosing a search strategy

The search axis is a strategy selection problem: how much of the optimization space to explore, who defines it, and how to trust the result (Wu *et al.*, 2024; Zheng *et al.*, 2020; Chen *et al.*, 2018a). When the team knows the schedule grammar well and wants fast, predictable compilation, a template or config-list search is right (Chen *et al.*, 2018a; OpenAI / triton-lang contributors, 2024). When the team needs reach beyond human templates and can afford compile time, a hierarchical sampler such as Ansor expands the space without leaving the schedule grammar (Zheng *et al.*, 2020). When the goal is discovering kernels no grammar encodes, a superoptimizer with verification is the only option, and its cost and confidence requirements must be accepted together (Wu *et al.*, 2024; Mirage Project contributors, 2025). The choice should be recorded with the deployment because it determines both the reachable performance and the maintenance burden of every future model (Chen, 2020; Davies *et al.*, 2025).

1.4.2 Reachability and verification correlation

The spectrum from human-specified to machine-discovered optimization carries a verification correlation that teams underweight (Wu *et al.*, 2024; Davies *et al.*, 2025). Schedules a human wrote can be trusted by review plus testing; schedules a search discovered need an independent correctness argument, which is why Mirage's probabilistic verifier is part of its design rather than an add-on (Wu *et al.*, 2024). Ansor's sampled schedules sit in a middle zone: they stay inside a grammar whose semantics are understood, so the verification burden is lighter than for a fully invented kernel (Zheng *et al.*, 2020; Chen, 2020). The deployment should match verification strength to discovery distance: the further the optimizer roams from human-vetted structure, the stronger the proof required before production (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Teams that adopt a discover-it-all optimizer while applying verify-by-testing discipline are taking the risk without the safety

mechanism (Wu *et al.*, 2024; Chen, 2026).

1.4.3 Search budget under model churn

Model churn is the adversary of expensive search, so the budget must be planned against the model-release cadence (Wu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). A team that ships a new model weekly cannot spend a week of superoptimization per release unless the hot regions are stable enough to cache across models (Wu *et al.*, 2024; Davies *et al.*, 2025). Search caches keyed on verified subgraph structure can amortize across releases, but cache hits must still pass verification against the new reference (Wu *et al.*, 2024; Chen, 2026). The practical policy is a graduated budget: cheap search for every model, moderate search for hot regions, and deep search only for the flagship model whose gains will be served longest (Chen, 2020; Davies *et al.*, 2025). Publishing the budget policy with the benchmark makes compile cost a predictable release metric rather than a surprise (Vellaisamy *et al.*, 2026; Chen, 2026). Model churn and search depth are therefore managed together, not separately (Wu *et al.*, 2024; Kwon *et al.*, 2023).

1.4.4 Cost metric conversions

Comparing search strategies requires converting their costs into the same units the deployment uses (Wu *et al.*, 2024; Chen *et al.*, 2018a; Davies *et al.*, 2025). Wall-clock compile time, on-device trial minutes, verification time, and artifact bytes all convert into release latency and infrastructure cost (Vellaisamy *et al.*, 2026; Chen, 2026). The conversion should be part of the search axis so a stack whose search is cheap in trials but expensive in human tuning effort is scored fairly (Chen *et al.*, 2018a; Wu *et al.*, 2024). A template search may require the team to write the templates, which is human cost invisible to a wall-clock timer; a superoptimizer converts that effort into machine time, which is visible but automatable (Chen *et al.*, 2018a; Wu *et al.*, 2024). Publishing the cost conversions with the benchmark lets a team pick the axis that matches its resources: engineers who can write templates, machines that can search, or products that can amortize either (Davies *et al.*, 2025; Chen, 2026).

1.5 YiRage benchmark

This is where the book's yardstick is defined. **YiRage** is the Mirage-lineage compiler used throughout the book to **benchmark** the others, and its discipline

is to measure every stack on the same two decode quantities—bytes/token and launches/token at batch-1—rather than each project's preferred microbenchmark (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). It generalizes Mirage's searched-and-verified megakernel across multiple backends (CUDA, CPU, Triton, and more) and carries the Chapter 2 residency rules as checkable IR metadata, so a candidate schedule that would spill is rejected before it ships (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The reason a common yardstick is necessary rather than pedantic is that the alternative—trusting each project's published number—systematically misleads. Ansor's 3.8×, AutoTVM's 1.2–3.8×, Mirage's 1.1–2.9×, and MPK's 1.2–6.7× are all real, but each is measured against a different baseline, on a different workload, often at a throughput-shaped operating point rather than batch-1 decode (Zheng *et al.*, 2020; Chen *et al.*, 2018a; Wu *et al.*, 2024; Mirage Project contributors, 2025). Reduced to one harness that fixes the model, the decode shape, and the metric, the numbers become comparable and the niches become visible: the megakernel path wins where launches dominate, the AOT bundle wins where the device is small, and the broad scheduler wins where portability is the constraint (Vellaisamy *et al.*, 2026; Chen, 2026).

Run as a **benchmark** harness, YiRage reframes the comparison as a routing problem: on a datacenter GPU it favors a searched megakernel path of the kind MPK demonstrates (1.2–6.7×); on an edge CPU it favors a Glow-style AOT bundle; on a broad portability target it favors IREE-style whole-program scheduling (Mirage Project contributors, 2025; Rotem *et al.*, 2018; IREE / iree-org contributors, 2024). The **cross-hardware** reading is that no single compiler dominates—each is the right answer for a region of the hardware matrix—and the value of a unified yardstick is that it makes the routing decision measurable instead of tribal (Davies *et al.*, 2025; Chen, 2026). Concretely, the routing inputs are the target class and the reuse profile of the workload: a high-reuse decode layer on a datacenter GPU routes to a searched megakernel because the one-time search amortizes and the launch collapse is largest there, while the same layer on a memory-constrained edge part routes to an AOT bundle because the binding fits the device and there is no launch loop to begin with (Mirage Project contributors, 2025; Rotem *et al.*, 2018). Encoding that decision as metadata rather than tribal preference is what lets the choice survive a model or hardware change without re-litigating it (Chen and YiRage Contributors, 2026).

1.5.1 Harness definition and shared artifacts

The unified harness needs a precise definition before any compiler runs: the reference model and checkpoint, the exact tokenizer, the batch-1 decode shape, the context buckets, the warm-up protocol, and the metric formulas for bytes/token and launches/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Every stack receives the same reference program and is asked to produce its best artifact under a documented budget, then all artifacts run through the same measurement code (Chen, 2020; Davies *et al.*, 2025). The artifacts themselves should be archived: compiled binaries, configuration files, compile reports, and verification outputs (Chen, 2026; Vellaisamy *et al.*, 2026). A benchmark whose artifacts cannot be replayed is a narrative, not a measurement (Davies *et al.*, 2025; Chen, 2026). The harness definition belongs in version control beside the compilers under test, so an upgrade to any component triggers a clean re-run rather than a silent comparison against stale numbers (Chen, 2020; Vellaisamy *et al.*, 2026).

1.5.2 Benchmark lifecycle and routing updates

The unified benchmark is not a one-time study; it is a living artifact that must be re-run when any input changes (Chen, 2026; Davies *et al.*, 2025). Model releases, compiler upgrades, driver updates, and new context buckets can each change the routing decision that the last benchmark produced (Vellaisamy *et al.*, 2026; Chen, 2020). The routing table should carry a timestamp and the benchmark commit that produced it, and a re-run should be scheduled on every relevant dependency change (Davies *et al.*, 2025; Chen, 2026). When a re-run changes a routing cell, the deployment should treat it as a configuration change with the same review as a flag change (Chen, 2020; Vellaisamy *et al.*, 2026). This lifecycle turns compiler selection from a one-time architecture decision into a maintained routing contract (Davies *et al.*, 2025; Chen, 2026).

1.5.3 Limits of the unified comparison

A common harness makes compilers comparable, but it does not make them equivalent (Davies *et al.*, 2025; Chen, 2026). Each stack still carries ecosystem constraints—framework integration depth, team skill, library maturity, and long-term maintenance—that no bytes/token measurement captures (Chen, 2020; Vellaisamy *et al.*, 2026). The unified numbers answer the narrow question of decode efficiency; the routing decision answers the broader question of production fit (Davies *et al.*, 2025; Chen, 2026). The benchmark should therefore publish both the measured

rows and the caveats, so a reader can apply the table without over-reading it (Chen, 2020; Vellaisamy *et al.*, 2026). Recognizing these limits is what keeps the yardstick honest rather than turning it into a religion (Davies *et al.*, 2025; Chen, 2026).

1.5.4 Report schema

Every benchmark row should carry the same schema so rows are comparable across time: model, checkpoint, compiler and version, target and SKU, context bucket, the three counters, compile time, verification status, and the artifact hash (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). The schema makes diffs between runs mechanical: a regression is a row whose counters changed under an unchanged schema (Chen, 2020; Davies *et al.*, 2025). It also makes the routing table auditable, because every routing decision can point at the row that justified it (Chen, 2026; Vellaisamy *et al.*, 2026). Publishing the schema as part of the harness forces each compiler project's measurement conventions into one comparable form instead of relying on their favored units (Davies *et al.*, 2025; Chen, 2026). A stable report schema is the smallest unit of benchmark governance that actually works (Chen, 2020; Vellaisamy *et al.*, 2026).

1.5.5 Routing table maintenance

The routing table is a map from workload and target cells to compiler choices, and maintaining it is an engineering task, not a one-time survey (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026). Each cell should name the chosen stack, the benchmark row that justified it, and the rollback cell if the choice fails in production (Chen, 2020; Vellaisamy *et al.*, 2026). When a workload moves between cells—say a model graduates from edge to datacenter serving—the routing change must be reviewed as a configuration change rather than applied silently (Davies *et al.*, 2025; Chen, 2026). The table should be generated from the benchmark repository so it cannot drift from the evidence (Chen, 2020, 2026). A living routing table is the operational form of the book's claim that compiler choice is a routing decision (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

1.5.6 Worked routing scenario

Consider one Llama-class model served on three cell types. On an H100 cell with steady traffic, the benchmark favors a searched megakernel because launch collapse dominates and the search amortizes over high reuse (Wu *et al.*, 2024; Mirage

Project contributors, 2025; Vellaisamy *et al.*, 2026). On a mid-tier GPU with mixed batch sizes, an IREE whole-program artifact may win because portability across the cell's shapes matters and dispatch collapse suffices (IREE / iree-org contributors, 2024; Chen, 2026). On a power-constrained edge box, a Glow-style AOT bundle wins because the device has no launch loop and memory is the binding constraint (Rotem *et al.*, 2018; AMD, 2024a). Each cell uses the same reference model and the same counters, so the three choices are comparable even though the mechanisms differ (Davies *et al.*, 2025; Chen, 2026). The scenario demonstrates why no single compiler dominates: the routing table is the actual deliverable, and the benchmark is its engine (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

1.5.7 Re-run policy for the yardstick

The unified benchmark needs an explicit re-run policy so its conclusions do not silently age (Davies *et al.*, 2025; Chen, 2026). Re-runs should be scheduled when the model changes, when a compiler under test releases a new version, when a driver or runtime is upgraded, and when the fleet adds a new device class (Vellaisamy *et al.*, 2026; Chen, 2020). Each re-run publishes the full report schema, so a routing change is always traceable to a row diff (Davies *et al.*, 2025; Chen, 2026). The policy should also define who owns the benchmark, because an unowned benchmark decays into folklore within two quarters (Chen, 2020; Vellaisamy *et al.*, 2026). A yardstick with a re-run policy is the difference between compiler routing as a maintained system and compiler routing as a stale slide (Davies *et al.*, 2025; Chen, 2026).

1.5.8 Documenting the machine model

The binding axis is only comparable when each compiler's machine model is documented alongside its rating (Wu *et al.*, 2024; OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024). The machine model states which hardware properties the compiler reasons about: shared memory, SM count, cache hierarchy, DMA edges, or tile SRAM (SemiAnalysis, 2024; AMD, 2024a). A compiler that omits a property from its model cannot optimize for it, and the rating should note that omission rather than penalize it blindly (Wu *et al.*, 2024; Chen, 2026). Documenting the machine model turns binding from an abstraction debate into a concrete capability list: this stack can reason about these resources and therefore can optimize this class of targets (OpenAI / triton-lang contributors, 2024; AMD, 2024b). The documentation belongs in the benchmark repository with

the device-class cells, so the capability claims are checkable (Davies *et al.*, 2025; Chen, 2026).

1.6 Key Takeaways

- **Rate compilers on launch collapse, not peak FLOPs.** At batch-1, launches/token and bytes/token predict ms/token; a stack can win an isolated kernel benchmark and lose end-to-end (Vellaisamy *et al.*, 2026; Chen, 2026). The launch-collapse column is the one that matters (NVIDIA, 2024).
- **Match compile-overhead model to deployment.** TVM/Triton pay JIT search; IREE/Glow pay AOT; Mirage pays superoptimization search (Zheng *et al.*, 2020; IREE / iree-org contributors, 2024; Wu *et al.*, 2024). Expensive one-time compilation amortizes over a reused decode layer but not a one-off graph (Kwon *et al.*, 2023).
- **Binding time trades breadth for depth.** Late-binding HALs (IREE) maximize portability but hide per-target residency; early, narrow bindings (Triton, Mirage) give up breadth for precise GPU reasoning (IREE / iree-org contributors, 2024; OpenAI / triton-lang contributors, 2024). A green build is not a green decode (Vellaisamy *et al.*, 2026).
- **Search reachability sets the ceiling.** Template search finds only anticipated schedules; a μGraph superoptimizer can invent megakernels (Chen *et al.*, 2018a; Wu *et al.*, 2024). What the search space contains bounds what the compiler can ever find (Zheng *et al.*, 2020).
- **No compiler dominates; routing does.** Each stack owns a region of the hardware matrix, so the win is a measured routing decision on common decode metrics—the YiRage discipline (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Megakernel on GPU, AOT bundle on edge, whole-program scheduling for breadth, and the routing itself recorded as metadata so it survives the next model or accelerator (Mirage Project contributors, 2025; Rotem *et al.*, 2018).

Practical decision rules. Five rules compress the comparison chapter for a design review. First, define the benchmark configuration before choosing the compiler, because a comparison without pinned variables measures nothing (Davies *et al.*, 2025; Chen, 2026). Second, weight the rating columns by your deployment constraints rather than by an average, because a leaderboard hides the columns

that matter (Vellaisamy *et al.*, 2026; Chen, 2020). Third, account for compile cost in the release pipeline and match it to the model's reuse profile (Wu *et al.*, 2024; Kwon *et al.*, 2023). Fourth, build the per-target residency proof for every cell, regardless of the binding time of the chosen stack (AMD, 2024a; Chen, 2026). Fifth, maintain the routing table as a living artifact that changes with the benchmark, not as a one-time architecture choice (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Teams that follow these rules make compiler selection measurable; teams that skip them repeat the religious debates this chapter exists to end (Chen, 2026; Vellaisamy *et al.*, 2026).

1.7 Conclusion

Compared on one honest axis—how few, how resident the kernel launches per token—the surveyed compilers stop looking like competitors and start looking like specialists: TVM and IREE for breadth, Triton for GPU kernel authoring, Glow for the edge, and Mirage for searched-and-verified megakernels (Chen *et al.*, 2018b; IREE / iree-org contributors, 2024; Tillet *et al.*, 2019; Rotem *et al.*, 2018; Wu *et al.*, 2024). The unifying contribution of a benchmark like YiRage's is to make that specialization measurable, so a serving team routes a workload to the stack whose niche it falls in rather than arguing microbenchmarks (Chen and YiRage Contributors, 2026; Chen, 2026). The practical payoff is that compiler selection stops being a one-time religious choice and becomes a per-workload, per-target decision backed by the same two numbers the whole book has tracked, which is the only form of comparison that survives the next model release or the next accelerator (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). With the compilers now rated on common ground, Chapter **??** turns from comparing stacks to driving one end to end: a compiler-driven performance-tuning workflow that takes a model from profile to shipped decode kernel (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 8, 9, 10, 16, 18

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 8, 16

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18

Chen, T., Zheng, L., Yan, E., Jiang, Z., Moreau, T., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018a). Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. AutoTVM; template-based statistical cost model (GBT/TreeGRU), transfer learning; 1.2–3.8x end-to-end. 5, 10, 11, 12, 13, 17

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018b). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 1, 2, 8, 18

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 4, 5, 6, 7, 9, 10, 11, 12, 14, 15, 16, 18

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 13, 15, 16, 17, 18

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 3, 8, 10, 11

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 1, 2, 5, 6, 8, 9, 10, 13, 16, 17, 18

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 3, 5, 6, 11, 12, 17, 18

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 3, 4, 5, 11, 13, 15, 17

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 8, 9, 10

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 2, 17

OpenAI / triton-lang contributors (2024). Triton: A language and compiler for custom deep-learning primitives. `https://github.com/triton-lang/triton`. MLIR-based backend: Triton IR (TTIR) → TritonGPU IR (TTGIR) → LLVM → PTX/AMDGCN; `@triton.autotune` with num_warps/num_stages. 2, 4, 5, 6, 7, 8, 10, 11, 16, 17

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 1, 2, 5, 8, 13, 16, 17, 18

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 8, 16

Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19. Tile = statically shaped sub-array; Triton-C + LLVM-based Triton-IR; matmul/conv on par with cuBLAS/cuDNN. 1, 2, 8, 18

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18

Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. (2020). Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879. Template-free auto-scheduler; hierarchical sampling, evolutionary search, task scheduler; up to 3.8x/2.6x/1.7x on Intel CPU/ARM CPU/NVIDIA GPU. 2, 3, 4, 5, 6, 7, 10, 11, 13, 17

Index

Compiler comparison, 1
Cross-hardware benchmark, 1